

SMF 工具 (SmfPlayer / MidiRecorder / TempoMapExtractor)

本页面介绍用于播放、记录和分析速度（Tempo）的 Standard MIDI File (SMF) 组件与工具类。

命名空间：

- 组件：`jp.kshoji.unity.midi`
- 工具类：`jp.kshoji.unity.midi.util`

位置：

- `Assets/MIDI/Scripts/Gameplay/SmfPlayer.cs`
- `Assets/MIDI/Scripts/Gameplay/MidiRecorder.cs`
- `Assets/MIDI/Scripts/MidiSequenceAsset.cs`
- `Assets/MIDI/Scripts/UmpSequenceAsset.cs`
- `Assets/MIDI/Scripts/Editor/UmpImportUtility.cs` / `UmpAssetImportMenu.cs`
- `Assets/MIDI/Scripts/Utilities/Smf/TempoMapExtractor.cs` 等

概述

组件 / 工具类	作用
MidiSequenceAsset	将 SMF 二进制数据作为 ScriptableObject 保存
UmpSequenceAsset	将 .midi2 （UMP MIDI Clip）二进制数据作为 ScriptableObject 保存
SmfPlayer	使用 SequencerImpl 播放 SMF / Sequence 并输出到 MidiManager
MidiRecorder	将实时 MIDI 输入记录为 SMF
TempoMapExtractor	从 Sequence 中按时间序列提取速度与拍号

以上组件均封装了既有的 `jp.kshoji.midisystem` （`Sequence` / `SequencerImpl`）。关于底层 SMF 输入输出 API 的详情，请参阅 [SMF / 序列化](#)。

MidiSequenceAsset

将 SMF 数据作为 Unity 资源保存的 ScriptableObject。用作 `SmfPlayer` 的播放源或 `MidiRecorder` 的保存目标。

创建方法

- Inspector：Assets > Create > MIDI > Sequence Asset
- 代码：`MidiSequenceAsset.CreateFromSequence(sequence, sourcePath)`

API

方法	说明
<code>SetSmfData(byte[] data, string sourcePath)</code>	设置 SMF 二进制数据
<code>ToSequence()</code>	转换为 <code>Sequence</code>
<code>CreateFromSequence(Sequence, string)</code>	从 <code>Sequence</code> 生成资源

UmpSequenceAsset

将 `.midi2`（UMP MIDI Clip / SMF2CLIP）二进制数据作为 Unity 资源保存的 ScriptableObject。用作 `UmpSequencer` 或 Scriptable Audio 的 `MidiDspUmpSequenceScheduler` 的播放源。

创建方法

- Inspector：Assets > Create > MIDI > UMP Sequence Asset（空资源）
- 从文件：Assets > Create > MIDI > Import UMP Sequence Asset From File（选择 `.midi2`）
- 代码：`UmpSequenceAsset.CreateFromSequence(umpSequence, sourcePath) / UmpImportUtility`

API

方法 / 属性	说明
<code>SetMidi2Data(byte[] data, string sourcePath, int index)</code>	设置 <code>.midi2</code> 二进制数据
<code>ToUmpSequence()</code>	将所设置的剪辑转换为 <code>UmpSequence</code>
<code>ToUmpSequenceList()</code>	转换文件内的所有剪辑
<code>CreateFromSequence(UmpSequence, string)</code>	从 <code>UmpSequence</code> 生成资源
<code>ClipIndex</code>	多剪辑文件内的索引（通常为 0）

在 Scriptable Audio 中的使用示例

```
var asset = /* 在 Inspector 中分配的 UmpSequenceAsset */;
var bootstrap = GetComponent<MidiDspUmpSequenceBootstrap>();
bootstrap.sequence = asset.ToUmpSequence();
bootstrap.EnsureReady();
bootstrap.Scheduler.Play();
```

关于 DSP 同步播放的详情，请参阅 [Unity 生态系统集成 — UMP 序列](#)。关于挂钟播放，请参阅 [MIDI 2.0 — UmpSequencer](#)。

SmfPlayer

将 `SequencerImpl` 集成到 Unity 生命周期中，并将 SMF 同步到游戏时间进行播放的组件。输出经由内部的 `MidiManagerReceiverBridge` 发送到 `MidiManager`。

设置

- 1. 将 `SmfPlayer` 挂载到 `GameObject` 上
- 2. 将 `MidiSequenceAsset` 分配到 `Sequence Asset`（或在运行时调用 `SetSequence()`）
- 3. 指定 `Output Device Id`（空 = 第一个输出设备）
- 4. 在场景 bootstrap 中调用 `MidiManager.InitializeMidi()`

MPTK 音频输出 (`FEATURE_USE_MPTK`)

若要输出到 MPTK 合成器而非硬件 MIDI 设备：

- 1. 在 Project Settings 中启用 `FEATURE_USE_MPTK`
- 2. 在同一 `GameObject` 上添加 `MptkSmfPlayerOutput`
- 3. 在 Play 模式下，`Configure()` 会将 `outputDeviceId` 设置为 `mptk:internal`

详情请参阅 [Maestro / MPTK 集成 — SmfPlayer 联动](#)。

编辑器 SMF 试听

在 `Window > MIDI > SMF Preview` 中读取 `.mid` 后，可从 **Preview Audio (MPTK)** 在 Play Mode 下试听（需要 `FEATURE_USE_MPTK`）。详情请参阅 [Maestro / MPTK 集成 — 编辑器 SMF 试听](#)。

主要设置

字段	说明
<code>sequenceAsset</code>	要播放的 SMF 资源
<code>group</code>	MIDI 组 (0–15)
<code>outputDeviceId</code>	输出设备 ID（空 = 自动选择）
<code>outputChannel</code>	将所有消息映射到此通道（-1 = 保持原始通道）
<code>playOnAwake</code>	Awake 后自动播放
<code>loop / loopCount</code>	循环播放（-1 = 无限）
<code>tempoBpm</code>	播放速度（运行中的变更也会反映）

操作 API

方法	说明
<code>Play()</code>	开始播放（暂停中会保持位置继续播放）
<code>Pause()</code>	暂停
<code>Stop()</code>	停止并返回开头
<code>Seek(float timeSeconds)</code>	跳转播放位置
<code>SetTrackMute(index, mute)</code>	轨道 Mute
<code>SetTrackSolo(index, solo)</code>	轨道 Solo
<code>SetSequenceAsset(asset)</code>	替换资源并重新加载
<code>SetSequence(sequence)</code>	在运行时直接指定 Sequence

状态属性

属性	说明
<code>State</code>	<code>Stopped</code> / <code>Playing</code> / <code>Paused</code>
<code>CurrentTimeSeconds</code>	当前的播放位置（秒）
<code>TotalDurationSeconds</code>	总播放时间（秒，基于 <code>TempoMapExtractor</code> ）

事件

UnityEvent	时机
<code>onPlaybackStarted</code>	开始播放
<code>onPlaybackPaused</code>	暂停
<code>onPlaybackStopped</code>	停止
<code>onPlaybackFinished</code>	在无循环的情况下到达末尾

使用示例

```
using jp.kshoji.unity.midi;
using UnityEngine;

public sealed class SmfPlaybackController : MonoBehaviour
{
    public SmfPlayer player;

    void Start()
    {
        MidiManager.Instance.InitializeMidi(() => player.Play());
    }

    public void OnSeek(float normalized)
    {
        player.Seek(player.TotalDurationSeconds * normalized);
    }
}
```

MidiRecorder

将实时 MIDI 输入（Note / CC / Program Change / Pitch Bend / Aftertouch / SysEx / 系统实时消息）记录到 `Sequence`，并保存为 `.mid` 文件或 `MidiSequenceAsset`。

现行实现采用基于 `Time.time` 的自有时间戳方式（与 `SequencerImpl` 录音 API 的集成属于未来扩展）。

设置

- 1. 将 `MidiRecorder` 挂载到 `GameObject` 上
- 2. 以 `registerWithMidiManager = true`（默认）向 `MidiManager` 注册
- 3. 根据需要设置 `deviceIdFilter` / `channelFilter`
- 4. 通过 `StartRecording()` / `StopRecording()` 控制记录

若通过过滤器链使用，请设置 `registerWithMidiManager = false`，并将其放置在上游过滤器的 `forwardTargets` 中（与 [面向游戏玩法的组件](#) 相同）。

主要设置

字段	说明
<code>tempoBpm</code>	记录时的速度（用于 Tick 计算）
<code>ticksPerQuarter</code>	分辨率（PPQ，默认 480）
<code>deviceIdFilter</code>	记录对象设备（空 = 所有设备）
<code>channelFilter</code>	记录对象通道（空 = 所有通道）

API

方法	说明
<code>StartRecording()</code>	开始记录
<code>StopRecording()</code>	停止记录
<code>ClearRecording()</code>	清除记录内容
<code>GetSequence()</code>	获取已记录的 <code>Sequence</code>
<code>SaveToFile(path)</code>	保存为 <code>.mid</code> 文件
<code>SaveToAsset(assetPath)</code>	创建 <code>MidiSequenceAsset</code> （仅 Editor）

记录格式

- SMF Format 1（速度轨道 + 数据轨道）
- 记录对象：Note On/Off、Control Change、Program Change、Pitch Bend、Channel/Poly Aftertouch、SysEx / System Common、MIDI 实时消息（Clock / Start / Stop 等）
- Raw UMP 不写出到 SMF（通过 `onMessage` 另行处理）

使用示例

```
using jp.kshoji.unity.midi;
using UnityEngine;

public sealed class RecordingExample : MonoBehaviour
{
    public MidiRecorder recorder;
    public SmfPlayer player;
    public MidiSequenceAsset recordedAsset;

    public void RecordAndPlay()
    {
        recorder.StartRecording();
        // ... MIDI 演奏 ...
        recorder.StopRecording();

#if UNITY_EDITOR
        recordedAsset = recorder.SaveToAsset("Assets/Recorded.mid.asset");
        player.SetSequenceAsset(recordedAsset);
        player.Play();
#endif
    }
}
```

TempoMapExtractor

从 `Sequence` 或 SMF 数据中按时间序列提取速度变更与拍号的静态工具类。可用于计算 `SmfPlayer.TotalDurationSeconds`，或作为谱面生成的基础。

API

```
using jp.kshoji.unity.midi.util;
using jp.kshoji.midisystem;

TempoMap map = TempoMapExtractor.Extract(sequence);
// 或
TempoMap map = TempoMapExtractor.Extract(smfBytes);
TempoMap map = TempoMapExtractor.Extract("/path/to/file.mid");
```

TempoMap 属性 / 方法

名称	说明
<code>TempoChanges</code>	速度变更列表
<code>TimeSignatures</code>	拍号列表
<code>TotalDurationSeconds</code>	总播放时间（秒）
<code>TotalTicks</code>	总 Tick 数
<code>TicksPerQuarter</code>	分辨率
<code>GetBpmAt(tick)</code>	指定 Tick 时点的 BPM
<code>TickToSeconds(tick)</code>	Tick → 秒
<code>SecondsToTick(seconds)</code>	秒 → Tick

对于不存在速度元事件的 SMF，将以 120 BPM（500000 μ s/qn）作为默认值处理。

使用示例

```
var map = TempoMapExtractor.Extract(sequence);
Debug.Log($"Duration: {map.TotalDurationSeconds:F2}s, BPM at 0: {map.GetBpmAt(0)}");

foreach (var signature in map.TimeSignatures)
{
    Debug.Log($"{{signature.TimeSeconds:F2}}s -> {{signature.Numerator}}/{{signature.Denom
```

MeasureTimeUtility

在 `TempoMap` 上进行小节 / 拍 Tick 计算，以及小节 ↔ 时间的转换。

位置：`Assets/MIDI/Scripts/Utilities/Smf/MeasureTimeUtility.cs`

方法	说明
<code>TicksPerBar(ppqn, numerator, denominator)</code>	一小节的 Tick 长度
<code>TicksForBars(...)</code>	N 小节的长度
<code>TicksPerBeat(...)</code>	简单拍号下的拍长（小节 / 分子）
<code>MeasureToStartSeconds</code> / <code>MeasureToEndSeconds</code>	从 1 开始的小节号 → 秒
<code>TryResolveMeasureRange</code> / <code>TryGetMeasureRangeMs</code>	两端包含的小节范围
<code>TimeSecondsToMeasure</code>	秒 → 从 1 开始的小节号

小节 Tick 公式：`ppqn * 4 * numerator / denominator`（整数除法）。

TupletUtility / SwingUtility

位置：Assets/MIDI/Scripts/Utilities/Smf/TupletUtility.cs, SwingUtility.cs

TupletUtility

固定 PPQN 网格上的有理连音（例如 3:2 的八分音符三连音）。步进 Tick 使用整数除法，因此范围内的 index→tick→index 可以往返。

方法	说明
UnitTicks / GroupDurationTicks / StepTicks	单位、组跨度与每步 Tick
TickAtIndex / IndexAtTick	双向映射
QuantizeToGroup	将 Tick 吸附到最近的连音步进
IsIndexTickReversible	验证所有索引的往返可行性

SwingUtility

播放时摇摆：在不改写编辑 Tick 的情况下延迟偶数弱拍。swingAmount ≈ 0.33 可得到类似三连音的感觉。

方法	说明
UnitTicksFromDivision(ppqn, division)	单位长度（4 = 四分、8 = 八分、...）
ApplySwing(editTick, unitTicks, amount)	编辑 Tick → 摇摆后的播放 Tick
ApplySwing(..., ppqn, swingDivision, amount)	便捷重载

MidiMetaMessageFactory

构建与读取常用的 SMF `MetaMessage` 载荷。

位置：`Assets/MIDI/Scripts/Utilities/Smf/MidiMetaMessageFactory.cs`

方法	说明
<code>CreateTempo(bpm)</code>	Set Tempo 元事件（每四分音符的 μ s）
<code>CreateTimeSignature(numerator, denominator, ...)</code>	Time Signature 元事件
<code>CreateTrackName(name)</code>	Track / Sequence Name（UTF-8）
<code>DenominatorToExponent</code> / <code>ExponentToDenominator</code>	SMF 的 2 的幂次转换
<code>TryReadTempoBpm(meta, out bpm)</code>	从 Tempo 元事件读取 BPM

源文件

文件	作用
MidiSequenceAsset.cs	SMF ScriptableObject
UmpSequenceAsset.cs	UMP .midi2 ScriptableObject
Editor/UmpImportUtility.cs / UmpAssetImportMenu.cs	.midi2 导入
SmfPlayer.cs / SmfPlayerState.cs	SMF 播放组件
MidiManagerReceiverBridge.cs	Sequencer → MidiManager 桥接
MidiRecorder.cs	录制组件
MidiRecordingSession.cs	录制会话（Tick 计算 / SMF 写出）
TempoMapExtractor.cs	速度映射提取
TempoMap.cs / TempoMapEntry.cs / TimeSignatureEntry.cs	数据模型
MeasureTimeUtility.cs	小节 / 小节 Tick 辅助
TupletUtility.cs / SwingUtility.cs	连音与摇摆时值
MidiMetaMessageFactory.cs	SMF 元事件构建

相关文档

- [SMF / 序列化 \(jp.kshoji.midisystem\)](#)
- [MIDI 2.0 / UMP](#)
- [Unity 生态系统集成 — UMP 序列](#)
- [面向游戏玩法的组件](#)
- [MIDI 1.0 运行时 \(MidiManager\)](#)
- [示例项目](#)